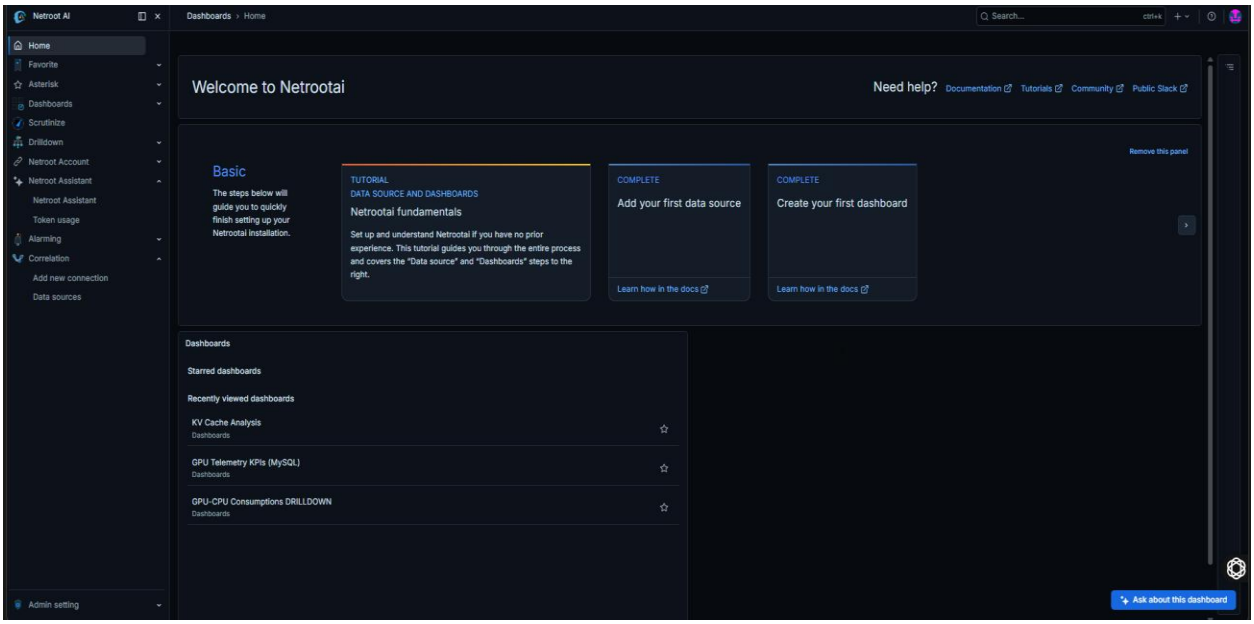
Netrootai -- Telemetry & Analysis Dashboard

Overview

The Netrootai Telemetry & Analysis Dashboard provides a consolidated view of model performance, GPU-related telemetry, KV-cache behavior, throughput, latency, and estimated infrastructure cost. It brings key indicators and time-series views into one workspace so users can review performance patterns and compare selected model and GPU configurations.



1.1 Key Capabilities

- **Multi-Source Data Integration:** Connect and unify telemetry from multiple Netrootai Analytics-compatible data sources, including SQL databases and time-series engines.
- **Flexible Infrastructure Scope:** Adaptable across compute workloads, covering CPU and GPU performance, memory pressure, and cluster network telemetry.
- **Interactive Dashboarding:** Enable rich visual exploration through adjustable time ranges, dynamic refresh intervals, entity-based filtering, and metric comparisons.

- **Statistical KPI Engine:** Support rolling statistical analysis, including mean, median, and standard deviation, using configurable sample windows to identify performance patterns and variations.
- **Custom Query Builder:** Enable users to write native SQL queries within Netrootai Analytics to calculate customized domain-specific metrics, filters, and aggregations.
- **Query Optimization:** Support efficient data retrieval through indexed telemetry schemas, targeted time-range filters, and optimized query execution.
- **Netrootai LLM Assistant:** Provide an integrated large language model (LLM)-powered assistant that helps users interpret dashboard information and understand telemetry reports in depth. The assistant translates complex metrics, charts, and performance patterns into clear, contextual explanations, helping users identify anomalies, investigate potential bottlenecks, and understand relationships between infrastructure and workload indicators.
- **Intelligent Root-Cause Analysis and Recommendations:** Analyze visible dashboard findings to suggest possible reasons for performance degradation, memory pressure, increased latency, or throughput fluctuations. The assistant can recommend precautionary measures and follow-up investigations, such as reviewing resource allocation, examining cache eviction patterns, evaluating batch sizes, and monitoring relevant KPIs. These recommendations support troubleshooting and operational decision-making and should be validated against telemetry evidence and infrastructure context.

1.2 Opening the dashboard

1. Open the Netrootai Telemetry & Analysis Dashboard using the URL supplied by your administrator or project team.
2. Sign in if your deployment requires authentication.
3. Open the relevant telemetry dashboard from the dashboard navigation.
4. Use the time-range control to select the period you want to inspect.
5. Use the dashboard refresh control to refresh the displayed panels.

The dashboard URL, access method, and authentication requirements depend on the deployment environment.

2. Understanding the Dashboard

The primary monitoring workspace relies on structured Netrootai Analytics visualization panels to render live telemetry. Users can track temporal patterns, perform entity-level benchmarking, and isolate anomalous telemetry spikes across connected clusters

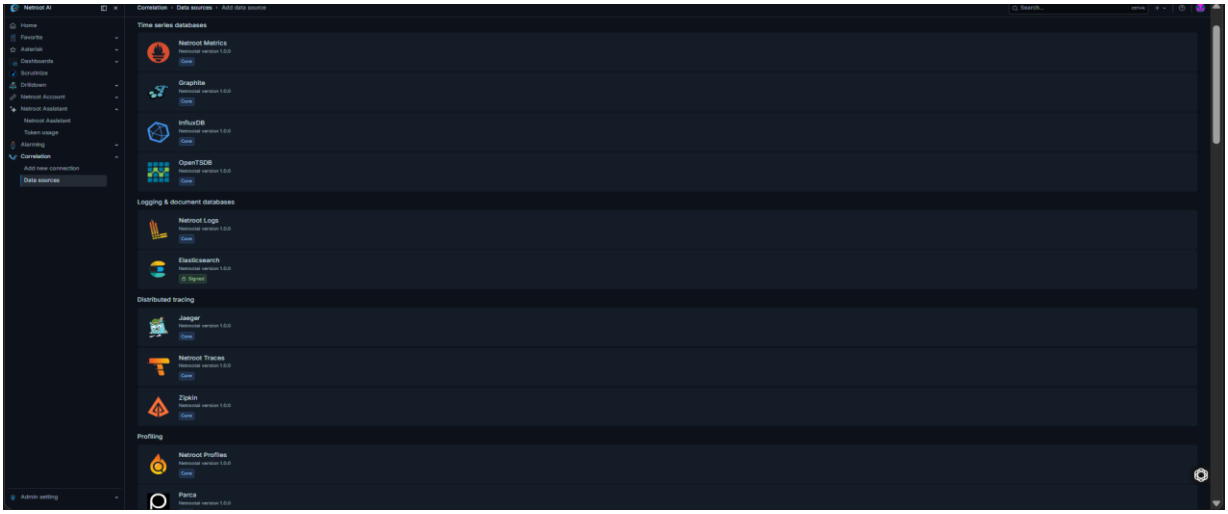
2.1 Time-Series Visualizations & 2.2 Multi-Machine Monitoring

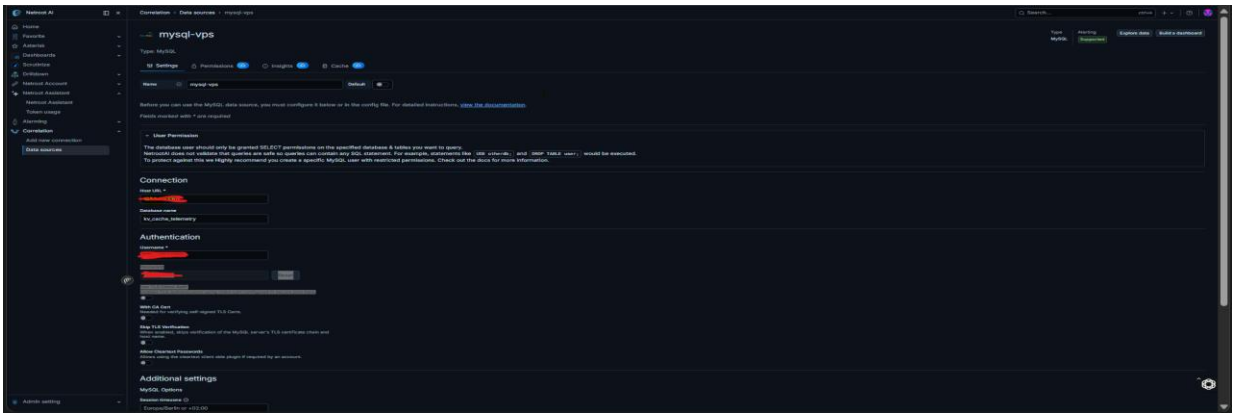
Time-series panels render telemetry readings against unified UTC timestamps to reveal utilization spikes and cyclical profiles. Monitored nodes (e.g., series m0, m1, m2) represent individual servers for side-by-side performance benchmarking.



3. Multiple Data-Source Integration

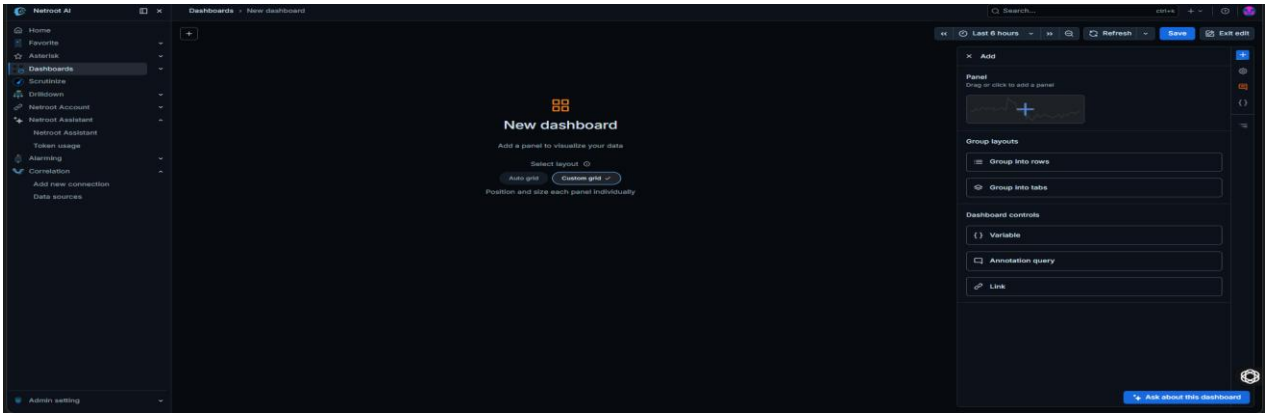
Netrootai supports simultaneous connections to heterogeneous backend data stores (PostgreSQL, MySQL, Prometheus, time-series engines) within a unified workspace. Users navigate to Connections → Data sources to configure connections.





4. Creating Custom KPIs in Netrootai Analytics

To support specialized operational needs, authorized users can author custom KPIs using native database queries directly within the panel query editor.



4.1 Code Example: Custom Average CPU KPI

```
SELECT
        machine_id,
        AVG(cpu_util) AS avg_cpu_utilization
FROM cluster_cpu_util
WHERE machine_id = 'm0'
GROUP BY machine_id;
```

4.2 Custom KPI Application Matrix

- Average CPU Utilization: Measures baseline processor compute loading across defined cluster groups.
- Peak GPU Utilization: Identifies maximum compute workload spikes on accelerator hardware.
- Memory Consumption: Tracks RAM/VRAM allocation trends to detect potential memory leaks.
- Resource Variance: Evaluates operational stability and workload predictability.
- Node Benchmark Comparison: Highlights load imbalance and non-uniform work distribution across nodes.
- Threshold Alerts: Triggers visual state changes when metrics breach critical capacity bounds.

5. Statistical Performance Indicators

Pre-built statistical indicators calculated over a calibrated 15-reading rolling window smooth noise and highlight genuine performance trends.

- Rolling Mean: Arithmetic average across 15 readings | Identify sustained capacity trends
- Rolling Median: 50th percentile (central value) | Filter transient telemetry noise
- Rolling Std Dev: Dispersion / variance relative to mean | Assess workload stability & volatility.

6. Query Optimization & Performance

- Range Filtering: Always bound queries using time macros.
- Column Selection: Select explicit metric columns rather than SELECT *.
- Indexing: Ensure composite indexes on (machine_id, timestamp) for rapid range scans.

7. What you can monitor

Depending on the panels enabled in your deployment, the dashboard can

include:

- **Mean KV-cache memory utilization (%)** --- overview of cache capacity usage.
- **Cache eviction rate (%)** --- rate of cache eviction events.
- **Time to First Token (TTFT)** --- time before the first output token is produced.
- **Time per Output Token (TPOT)** --- time associated with producing subsequent output tokens.
- **Throughput (tokens/sec)** --- token generation throughput.
- **KV-cache compression ratio** --- compression achieved for cache data.
- **Latency overhead (%)** --- additional latency associated with the measured configuration.
- **Estimated GPU cost per minute ($)** --- estimated cost indicator for the selected configuration.
- **KV-cache memory by model** --- comparison of memory consumption across models.
- **Dynamic memory pressure heatmap** --- visual overview of memory-pressure patterns across displayed time points and series.
- **KV-cache memory utilization over time** --- time-series view of utilization changes.

- **Throughput by batch size** --- comparison of throughput across batch sizes.
- **KV-cache memory vs. throughput** --- view of the relationship between memory consumption and throughput.
- **Model and GPU comparison table** --- tabular view of metrics for selected model/GPU combinations.

NOTE - Custom KPI Flexibility:

Users are not limited to the predefined KPIs available on the dashboard. They can define, build, and customize their own KPIs based on specific business requirements, operational objectives, and analytical needs using custom queries, metrics, filters, and aggregations.

8. Using the model and GPU filters

The dashboard provides model and GPU selectors at the top.

8.1 Select a model

1. Open the **Select AI Model** filter.
2. Choose one or more available model names.
3. Apply the selection if the interface requires confirmation.
4. Review the KPI cards, charts, and table for the selected models.

8.2 Select a GPU

1. Open the **Select GPU** filter.
2. Choose one or more available GPU types.
3. Apply the selection if required.
4. Review the updated dashboard panels.

8.3 Compare configurations

1. Select the models you want to inspect.
2. Select the GPU types you want to compare.
3. Review the KPI cards for a quick summary.

4. Use the model-level charts and comparison table to inspect the available breakdown.
5. Change one filter at a time when investigating differences between configurations.

The available combinations depend on the records and configuration exposed by the connected telemetry data source.

9. Understanding the panels

9.1 KPI cards

KPI cards provide a quick summary of individual metrics. Check the displayed unit before comparing values. For example, KV-cache memory in GB and KV-cache utilization in percent measure different things and should not be expected to have the same numeric value.



9.2 Time-series panels

Time-series panels show how a metric changes across the selected time range. Use the time-range selector and refresh controls to review the period of interest. The displayed time range and refresh interval are controlled by dashboard settings.



9.3 Comparison charts

Comparison charts help identify differences between models, GPU configurations, batch sizes, or other dimensions exposed by a panel.

Check axis labels and units before drawing conclusions.



9.4 Model and GPU comparison table

The comparison table presents available metrics by model and GPU configuration.

1. Set the model and GPU filters first.
2. Confirm that the table contains the configurations you intended to review.
3. Read each value alongside its column heading and unit.
4. Use the timestamp or time-range context where available.
5. When comparing a table value with a KPI, verify that both panels use the same metric definition, filters, time range, and aggregation method. Values can differ when panels use different sources or calculations.



Model Deployment	GPU Architecture	Avg TTFT / TPOT	KV Cache Utilization
Llama-3-70B-Instruct	8x NVIDIA H100	120ms / 14ms	82% (Low Eviction)
Mistral-Large-2	4x NVIDIA A100 80GB	210ms / 22ms	94% (Moderate Pressure)
DeepSeek-Coder-33B	2x NVIDIA RTX 6000	180ms / 18ms	76% (Stable)

10. Dynamic dashboard behavior and rolling time windows

The dashboard supports interactive filtering and time-window

exploration:

- **Model and GPU filters:** Selecting one or more model names and GPU types updates panels configured to use these dashboard variables.
- **Rolling time window:** Time-series panels can display a moving time window. As time advances and the dashboard refreshes, the plotted window can move forward instead of remaining tied to a fixed historical interval.
- **Automatic refresh:** When enabled, the dashboard periodically re-runs panel queries. Users can also trigger a manual refresh.
- **Time-range controls:** The time-range selector lets users inspect a different period. The selected time range and each panel's query determine what is displayed.
- **Panel-specific behavior:** Some panels show time-series values, while others show a KPI or model/GPU comparison. Filters affect panels whose queries are configured to consume those variables.



10.1 Understanding rolling windows

A rolling window is a moving view of a recent period. For example, a panel configured to show the previous 60 minutes can display a sequence of points across that interval. When refreshed later, the window can advance to include a newer period.

The refresh control re-runs queries; it does not itself create or ingest telemetry records. The data source and each panel's query determine whether new points or updated values are available.

10.2 Dynamic filters and metric consistency

Model and GPU selectors narrow the configurations shown. For meaningful comparisons, confirm that each panel uses the intended filters and

compatible metric definitions. KPI cards, time-series charts, and the comparison table may use different query logic; values should be compared directly only when the source, filters, time window, and calculation are aligned.

11. Refreshing and time ranges

- Use the **time-range selector** to choose the period to inspect.
- Use the **refresh control** to request an updated view.
- An automatic refresh interval, when enabled, determines how often panels are queried again.
- A refresh requests the latest information available through the configured data source; it does not by itself guarantee that new telemetry records have been ingested.
- If the dashboard appears unchanged, verify the selected time range, refresh interval, data-source connection, and availability of recent records.

12. Local access and environment setup

The exact local startup command depends on the repository and deployment configuration used by your team. Use the project's existing environment configuration and startup instructions rather than creating a separate instance with different settings.

Before running the dashboard locally, confirm:

- The required runtime and dependencies are installed.
- The dashboard service is configured for the intended local address and port.
- The telemetry database is reachable from the local environment.
- The configured data-source connection uses the correct host, port, database, and credentials.

- Required network access and database permissions are in place.
- Secrets and passwords are supplied through the approved environment or secret-management mechanism, not committed to source control.

After startup, open the configured local address in a browser and follow

the **Opening the dashboard** and **Using the model and GPU filters**

sections above.

Configuration note: Hostnames, ports, credentials, and startup

commands vary by deployment. Use the values maintained by your project

Team.

13. Netrootai Intelligent LLM Assistant

The **Netrootai LLM Assistant** is an integrated, LLM-powered analytics companion that transforms complex telemetry dashboards into understandable, actionable insights. It helps users move beyond monitoring individual metrics to understanding performance patterns, investigating potential problems, and making informed operational decisions.

13.1 Key Highlights

1. Contextual Dashboard Interpretation

Interprets dashboard KPIs, charts, and telemetry patterns to provide clear, contextual explanations of system and workload performance. Helps users understand complex metrics, identify anomalies, and explore relationships between infrastructure indicators.

2. Intelligent Root-Cause Analysis

Examines visible performance patterns to suggest possible causes of memory pressure, cache eviction spikes, increased latency, throughput fluctuations, and other potential bottlenecks.

3. Precautionary Measures and Recommendations
Provides practical recommendations for further investigation and potential optimization,

including reviewing resource allocation, examining cache eviction behavior, evaluating batch sizes, and monitoring relevant performance indicators.

4. Conversational Analytics

Allows users to ask natural-language questions about dashboard findings, request detailed explanations, investigate specific models or GPU configurations, and explore performance-related questions without manually interpreting every chart.

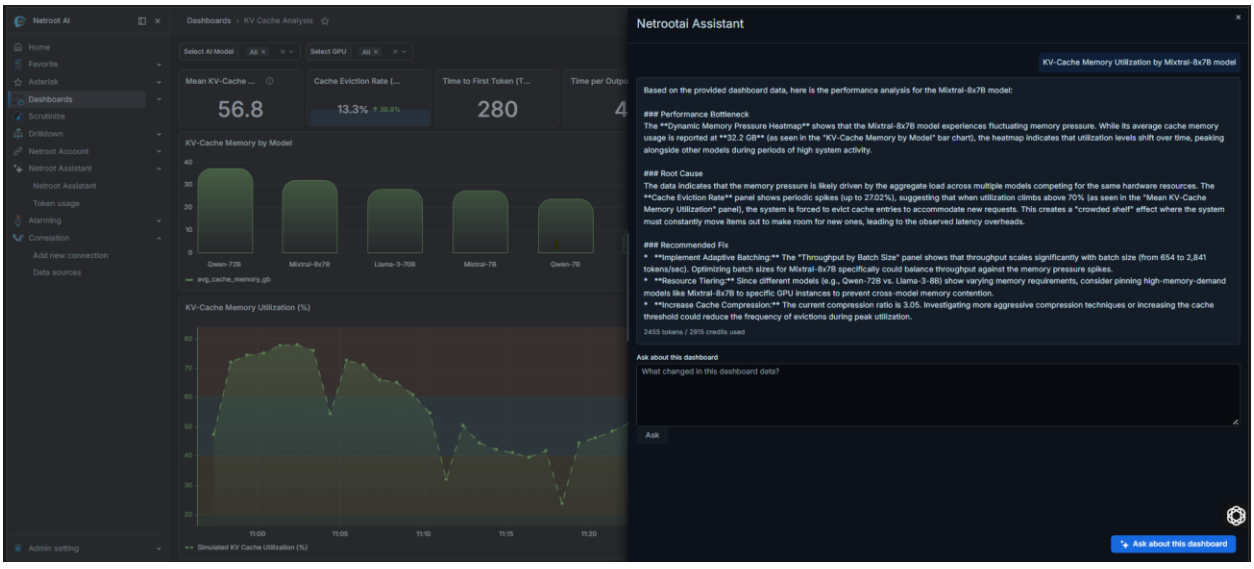
5. Decision Support for Infrastructure Optimization

Connects observations across relevant dashboard metrics to help users prioritize investigations, understand potential performance trade-offs, and identify opportunities to improve resource utilization and workload efficiency.

13.2 Business Value

The Netrootai LLM Assistant bridges the gap between telemetry visualization and actionable understanding. By combining conversational interaction with contextual performance analysis, it helps simplify troubleshooting, accelerate investigations, and support more informed infrastructure and workload optimization decisions.

Note: Root-cause explanations and recommendations are analytical guidance and should be validated against the underlying telemetry and operational context.



14. ⭐ GPU Inference Intelligence — Model vs. Hardware vs. Cost vs. Power

Netrootai Analytics helps users understand how AI model characteristics, GPU hardware, inference performance, operational costs, and power consumption interact. By bringing relevant telemetry and performance indicators into a unified analytical view, the platform helps users evaluate infrastructure configurations and make more informed deployment decisions.

1. Model-Level Analysis

Understand how different AI models behave during inference by examining:

- KV-cache memory requirements and utilization.
- Cache eviction patterns and compression ratios.
- Time to First Token (TTFT) and Time per Output Token (TPOT).
- Token throughput under different workload conditions.

This helps users investigate which models place greater demands on memory resources and how those demands relate to inference performance.

2. GPU Hardware Comparison

Compare supported GPU configurations using available telemetry to investigate:

- Memory capacity and utilization.
- Throughput and latency differences.
- Resource pressure and cache behavior.
- Potential hardware bottlenecks under different workloads.

This helps users assess whether a hardware configuration is appropriate for a particular model and workload.

3. Cost and Operational Efficiency

Use throughput, estimated GPU cost, and other available performance metrics to evaluate the relationship between inference performance and expenditure.

Potential analyses include:

- Estimated cost per unit of inference throughput.
- Cost differences between model–GPU configurations.
- Performance trade-offs associated with resource allocation.
- Opportunities to improve utilization and reduce unnecessary expenditure.

Cost-efficiency calculations depend on the availability and definition of cost and throughput measurements.

4. Power and Energy Awareness

Where GPU power and energy telemetry is available, analyze:

- GPU power consumption under different inference workloads.
- Energy consumption over a defined measurement interval.
- Relationships between power usage, throughput, and latency.
- Energy efficiency, such as tokens generated per joule.
- Potential opportunities to reduce energy consumption while maintaining performance requirements.

Power and energy conclusions require corresponding measurements from the infrastructure or a supported telemetry source. They should not be inferred from GPU utilization or estimated cost alone.

5. Cross-Dimensional Decision Support

The platform helps users investigate the trade-offs between **model requirements, GPU capabilities, inference performance, cost, and power consumption**.

For example, users can investigate whether a higher-throughput GPU configuration justifies its additional operating cost, whether KV-cache pressure is contributing to performance degradation, or whether another model–hardware combination offers a more suitable balance of latency, throughput, and resource usage.

Role of the Netrootai LLM Assistant

The Netrootai LLM Assistant complements these analyses by explaining the relationships between visible dashboard metrics in natural language. Users can ask questions such as:

- Which model–GPU combination provides the best balance of throughput and estimated cost?
- How might KV-cache pressure be related to increased inference latency?
- What could explain a throughput decrease when GPU memory utilization is high?
- Which additional power or energy metrics would help assess energy efficiency?
- What precautions should be considered before changing the model, batch size, or GPU allocation?

The assistant can explain observed patterns, suggest possible causes, and recommend follow-up investigations. Its conclusions should be validated against the available telemetry and operational context.

Business Value

By connecting model behavior, hardware performance, cost, and power-related telemetry, Netrootai Analytics helps engineering and infrastructure teams make better-informed decisions about inference deployment, resource allocation, operational efficiency, and infrastructure planning.

15. Troubleshooting

15.1 The dashboard does not load

- Confirm that the dashboard service is running.
- Verify the URL, port, network access, and authentication requirements.
- Review service logs for startup or configuration errors.

15.2 A panel reports a query or data-source error

- Confirm that the configured data source is available.
- Check the database name, table names, column names, and permissions.
- Review the panel query for syntax errors and confirm that referenced dashboard variables exist.

15.3 A model or GPU selection returns no rows

- Confirm that the selected model and GPU names are available in the connected data source.
- Try selecting one model and one GPU at a time.
- Check whether the selected combination has records in the chosen time range.

15.4 KPI values do not match the comparison table

Values match only when the KPI and table use the same metric definition, source, filters, time range, and calculation. Check whether one panel uses a stored database field while another uses a separate calculation, and whether the panels use different rounding or aggregation rules.

Align those definitions before treating the values as directly comparable.

15.5 Values do not appear to update

- Trigger a manual refresh.
- Check the dashboard's automatic refresh interval.
- Confirm that the selected time range includes the relevant records.
- Verify that the upstream telemetry process is delivering updated records to the configured data source.

15.6 A metric has an unexpected unit or sign

- Check the panel title, axis label, and unit configuration.
- Verify the field mapping and calculation used by the panel.
- Confirm that the metric definition and expected range are documented for the deployment.

16 Good practices

- Keep model and GPU selections explicit when capturing screenshots or sharing findings.
- Record the selected time range when reporting a result.
- Compare metrics only when their units and definitions are aligned.
- Use the table for configuration-level inspection and charts for patterns over time.
- Avoid interpreting a displayed value without checking its unit and panel definition.
- Report persistent errors with the panel name, selected filters, time range, and relevant error message. Never include passwords, tokens, or other secrets in a report.

17. Support

For access issues, environment setup, missing configurations, or unexpected telemetry results, contact the Netrootai project or platform administrator. Include the dashboard panel name, selected model/GPU filters, time range, and relevant error details.